Data Mining and Analytics (18CSE355T)

UNIT- 2 NOTES

What is Frequent Pattern Mining?

Frequent Pattern Mining (AKA Association Rule Mining) is an analytical process that finds frequent patterns, associations, or causal structures from data sets found in various kinds of databases such as relational databases, transactional databases, and other data repositories. Given a set of transactions, this process aims to find the rules that enable us to predict the occurrence of a specific item based on the occurrence of other items in the transaction.

Let's look at an example of Frequent Pattern Mining. First, we will want to understand the terminology used in this type of analysis. While there are numerous metrics and factors used in this technique, for this example, we will only consider two factors namely, Support and Confidence.

Support: T he support of a rule x -> y (where x and y are each items/events etc.) is defined as the proportion of transactions in the data set which contain the item set x as well as y. So, Support (x -> y)= no. of transactions which contain the item set x & y / total no. of transactions.

Confidence: The confidence of a rule x -> y is defined as: Support (x -> y) / support (x). So, it is the ratio of the number of transactions that include all items in the consequent (y in this case), as well as the antecedent (x in this case) to the number of transactions that include all items in the antecedent (x in this case).

In the table below, Support (milk->bread) = 0.4 means milk and bread are purchased together occur in 40% of all transactions. Confidence (milk->bread) = 0.5 means that if there are 100 transactions containing milk then there will be 50 that will also contain bread.

TID	Milk	Bread	Butter	Beer
1	1	0	1	1
2	1	1	1	0
3	0	1	1	0
4	1	0	0	1
5	1	1	1	1

How Does Frequent Pattern Mining Support Business Analysis?

This method of analysis can be useful in evaluating data for various business functions and industries.

- **Basket Data Analysis**: To analyze the association of purchased items in a single basket or single purchase.
- **Cross Marketing and Selling**: To work with other businesses that complement your own, not competitors. For example, vehicle dealerships and manufacturers have cross marketing campaigns with oil and gas companies for obvious reasons.
- **Catalog Design**: The selection of items in a business' catalog are often designed to complement each other, so that buying one item will lead to buying another, so these items are often complements or closely related.
- **Medical Treatments**: Each patient is represented as a transaction containing the ordered set of diseases, and which diseases are likely to occur simultaneously/ sequentially can be predicted.

To understand the value of this applied technique, let's consider two business use cases.

Use Case One

Business Problem: A retail store manager wants to conduct Market Basket analysis to come up with a better strategy of product placement and product bundling.

Business Benefit: Based on the rules generated, the store manager can strategically place the products together or in sequence leading to growth in sales and, in turn, revenue of the store. Offers such as "Buy this and get this free" or "Buy this and get % off on this" can be designed based on the rules generated.

Use Case Two

Business Problem: A bank-marketing manager wishes to analyze which products are frequently and sequentially bought together. Each customer is represented as a transaction, containing the ordered set of products, and which products are likely to be purchased simultaneously/sequentially can then be predicted.

Business Benefit: Based on the rules generated, banking products can be cross-sold to each existing or prospective customer to drive sales and bank revenue. For example, if savings, personal loan and credit cards are frequently/sequentially bought, then a new saving account customer can be cross-sold with a personal loan and credit card.

Frequent Pattern Mining (AKA Association Rule Mining) is an analytical process that finds frequent patterns, associations, or causal structures from data sets found in various kinds of data repositories. This method of analysis can be useful in evaluating data for various business functions and industries and is useful in determining the frequent patterns in buying behavior for various products and services, and in analyzing the relationships among various data points to cross-sell and bundle products, and service offerings, and to understand target audiences.

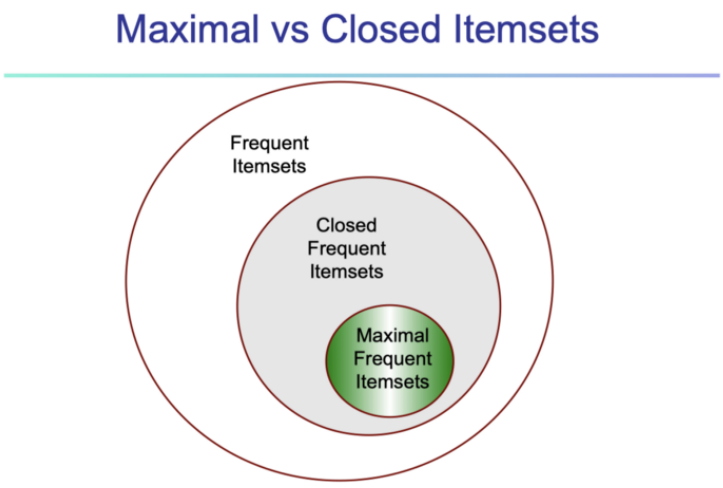
What are Closed and Maximal Frequent Itemsets

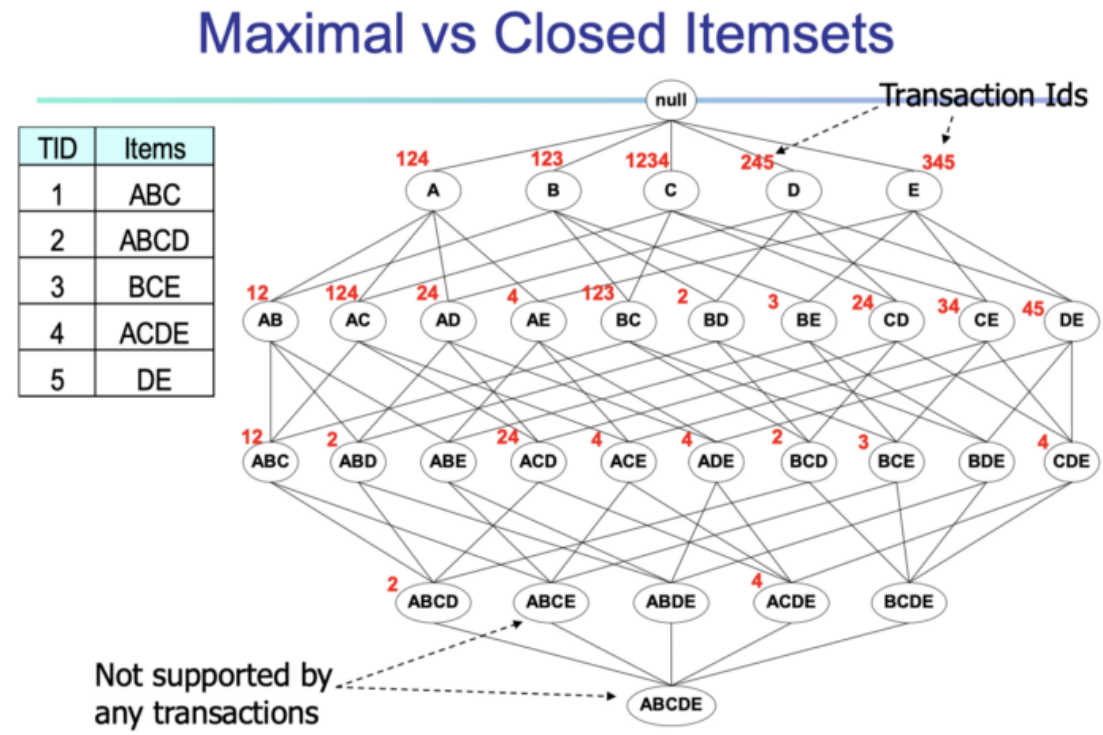
An itemset is whose support is greater than or equal to a minsup(Minimum Support) threshold. Support is the frequency of occurrence of an itemset. For example, given a set of transactions T, we would like to find all itemsets that appear more than 2 times in all transactions. This can be viewed as finding all frequent itemsets with minsup ≥2.

Then what are closed and maximal frequent itemsets?

By definition, An itemset is maximal frequent if none of its immediate supersets is frequent. An itemset is closed if none of its immediate supersets has the same support as the itemset. Let's use an example and diagram representation to better understand the concept.

Here we have transactions T with 5 transactions, let's present all the itemsets hierarchy using a tree-type diagram and write down the transactions the itemset appears on top in red.





Example demonstration.: If we set the minsup to be 2, any itemsets that appear more than twice will be frequent itemsets. And among those frequent itemsets, we can find closed and maximal frequent itemsets by comparing their support(frequency of occurrence) to their supersets.

Solved Example to Find out Frequent , closed and maximal items given below:-

TID	List of Items
T1	A, B, C, D
T2	A, B,C, D
T3	A,B,C
T4	B, C, D
T5	C,D

Min Support Count=3,

Find Frequent, Closed, Maximal Itemset

Item	Count
A	3
B	4
C	5
D	4

All items that is A, B, C & D are **frequent** because they sup count is greater than or equal to minimum support count.

Item	Count
A, B	3
A, C	3
A, D	2
B, C	4
B, D	3
C, D	4

All 2 items sets are frequent, except AD

A(3) → AB(3) , AC(3), AD(2)

A(count) is not greater than its immediate superset.

A is not closed.

In A's immediate superset, itemset are present with min. support count i.e. 3.

A is not maximal

B(4) → AB(3) , BC(4), BD(3)

B(count) is not greater than its immediate superset.

B is not closed.

In B's immediate superset, itemset are present with min. support count i.e. 3.

B is not maximal

Item	Count
A, B	3
A, C	3
A, D	2
B, C	4
B, D	3
C, D	4

C(count) is greater than its immediate superset.

C is closed.

In C's immediate superset, itemset are present with min. support count i.e. 3.

C is not maximal

D(4) → AD(2) , BD(3), CD(4)

D(count) is not greater than its immediate superset.

D is not closed.

In D's immediate superset, itemset are present with min. support count i.e. 3.

D is not maximal

Item	Count
A, B,C	3
A, B,D	2
A, C,D	2
B,C,D	3

AB(3) → ABC(3) , ABD(2)

AB(count) is not greater than its immediate superset.

AB is not closed.

In AB's immediate superset, itemset are present with min. support count i.e. 3.

AB is not maximal

AC(3) → ABC(3) , ACD(2)

AC(count) is not greater than its immediate superset.

AC is not closed.

In AC 's immediate superset, itemset are present with min. support count i.e. 3.

AC is not maximal

AD(2) → ABD(2) , ADC(2)

AD Not frequent

Item	Count
A, B,C	3
A, B,D	2
A, C,D	2
B,C,D	3

BC(4) → ABC(3) , BCD(3)

BC(count) is greater than its immediate superset.

BC is closed.

In BC's immediate superset, itemset are present with min. support count i.e. 3.

BC is not maximal

BD(3) → ABD(2) , BCD(3)

BD(count) is not greater than its immediate superset.

BD is not closed.

In AC 's immediate superset, itemset are present with min. support count i.e. 3.

BD is not maximal

CD(4) → ACD(2) , BCD(3)

CD(count) is greater than its immediate superset.

CD is closed.

In CD's immediate superset, itemset are present with min. support count i.e. 3.

CD is not maximal

Item	Count
A, B,C, D	2

ABCD is not frequent.

ABC(3) → ABCD(2)

ABC(count) is greater than its immediate superset.

ABC is closed.

In ABC's immediate superset, itemset are not present with min. support count i.e. 3.

ABC is maximal

ABD(2) → Not Frequent

ACD(2)→ Not Frequent

BCD(3) → ABCD(2)

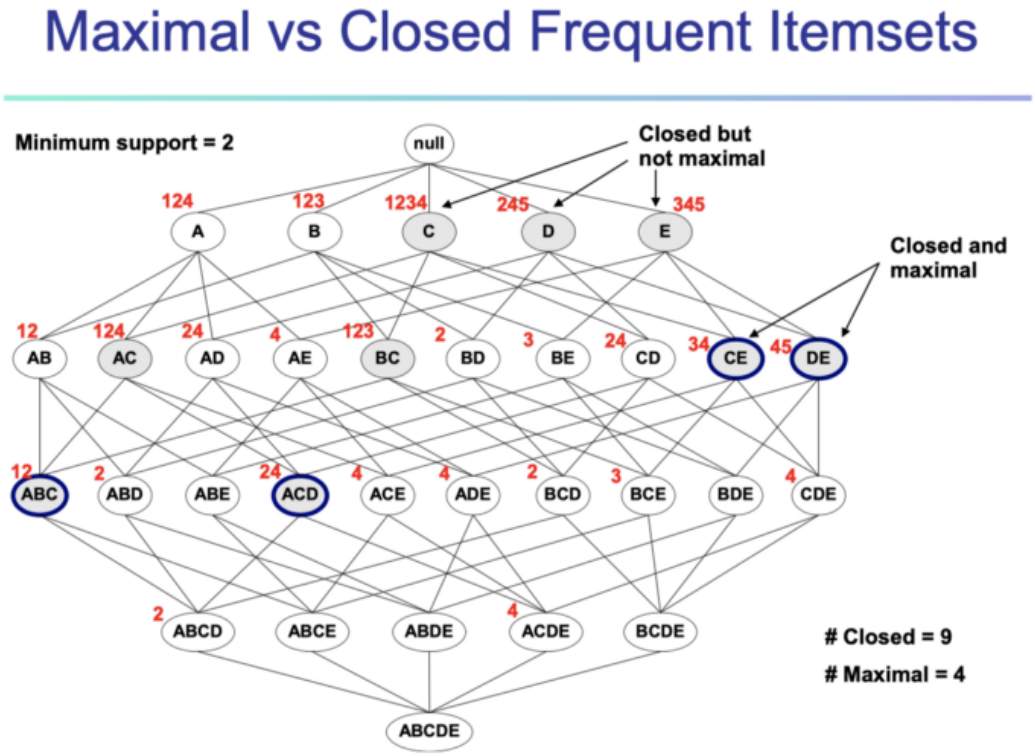
BCD(count) is greater than its immediate superset.

BCD is closed.

In BCD's immediate superset, itemset are not present with min. support count i.e. 3.

BCD is maximal

ABCD (2)→not frequent.



Highlight all Closed and Maximal Frequent Itemsets

We can see the maximal itemsets are a subset of closed itemsets. Also, the maximal itemset works as a boundary, anything below the maximal sets is not frequent itemsets(any superset of maximal itemsets are not frequent).

WHAT IS MARKET BASKET ANALYSIS?

In today's world, the goal of any organization is to increase revenue. Can this be done by pitching just one product at a time to the customer? The answer is a clear **no**. Hence, organizations began mining data related to frequently bought items.

Market Basket Analysis is one of the key techniques used by large retailers to uncover associations between items. They try to find out associations between different items and products that can be sold together, which gives assisting in right product placement. Typically, it figures out what products are being bought together and organizations can place products in a similar manner. Let's understand this better with an example:

People who buy Bread usually buy Butter too. The Marketing teams at retail stores should target customers who buy bread and butter and provide an offer to them so that they buy the third item, like eggs.

So if customers buy bread and butter and see a discount or an offer on eggs, they will be encouraged to spend more and buy the eggs. This is what market basket analysis is all about.

This is just a small example. So, if you take 10000 items data of your Supermart to a Data Scientist, Just imagine the number of insights you can get. And that is why Association Rule mining is so important.

How does market basket analysis work?

Market Basket Analysis is modeled on Association rule mining, i.e., the IF {}, THEN {} construct.

For example, IF a customer buys bread, THEN he is likely to buy butter as well.

Association rules are usually represented as: {Bread} -> {Butter}

Some terminologies to familiarise yourself with Market Basket Analysis are:

- **Antecedent:** Items or 'itemsets' found within the data are antecedents. In simpler words, it's the IF component, written on the left-hand side. In the above example, bread is the antecedent.
- **Consequent:** A consequent is an item or set of items found in combination with the antecedent. It's the THEN component, written on the right-hand side. In the above example, butter is the consequent.

TYPES OF MARKET BASKET ANALYSIS

Now that we have a fair idea of what is Market Basket Analysis and some of the key terms associated with an MBA, let us dig deeper. Market Basket Analysis techniques can be categorized based on how the available data is utilized:

1. **Descriptive market basket analysis**: This type only derives insights on past data and is the most frequently used approach. The analysis here does not make any predictions but simply rates the association between products using statistical techniques. For those familiar with the basics of Data Analysis, this type of modeling is known as unsupervised learning. Check Jigsaw Academy's **Data Science Bootcamp** for more information.
2. **Predictive market basket analysis**: This type uses supervised learning models like classification and regression. It essentially aims to mimic the market to analyze what causes what to happen. For example, buying an extended warranty is more likely to follow the purchase of an iPhone. Essentially, it considers items purchased in a sequence to determine cross-selling. While it isn't as widely used as a descriptive MBA, it is still a very valuable tool for marketers.
3. **Differential market basket analysis**: This type of analysis is a beneficial tool for competitor analysis. It compares purchase history between stores, between seasons, between two time periods, between different days of the week, etc., to find interesting patterns in consumer behavior. For example, it can help determine why some users prefer to purchase the same product with the same price on Amazon vs. Flipkart – the answer

can simply be that the Amazon reseller has more warehouses and can deliver faster, or maybe something more profound like user experience.

Association Rule Mining

Association rules can be thought of as an IF-THEN relationship. Suppose item **A** is being bought by the customer, then the chances of item **B** being picked by the customer too under the same **Transaction ID** is found out.

There are two elements of these rules:

Antecedent (IF): This is an item/group of items that are typically found in the Itemsets or Datasets.

Consequent (THEN): This comes along as an item with an Antecedent/group of Antecedents.

But here comes a constraint. Suppose you made a rule about an item, you still have around 9999 items to consider for rule-making. This is where the Apriori Algorithm comes into play. So before we understand the Apriori Algorithm, let's understand the math behind it. There are 3 ways to measure association:

- Support
- Confidence
- Lift

Support: It gives the fraction of transactions which contains item A and B. Basically Support tells us about the frequently bought items or the combination of items bought frequently.

$$Support = \frac{freq(A,B)}{N}$$

So with this, we can **filter out** the items that have a **low frequency**.

Confidence: It tells us how often the items A and B occur together, given the number times A occurs.

$$Confidence = \frac{freq(A,B)}{freq(A)}$$

Typically, when you work with the Apriori Algorithm, you define these terms accordingly. **But how do you decide the value?** Honestly, there isn't a way to define these terms. Suppose you've assigned the support value as 2. What this means is, until and unless the item/s frequency is not 2%, you will not consider that item/s for the Apriori algorithm. This makes sense as considering items that are bought less frequently is a waste of time.

Now suppose, after filtering you still have around 5000 items left. Creating association rules for them is a practically impossible task for anyone. This is where the concept of lift comes into play.

Lift: Lift indicates the strength of a rule over the random occurrence of A and B. It basically tells us the strength of any rule.

$$Lift = \frac{Support}{Supp(A)\ x\ Supp(B)}$$

- Focus on the denominator, it is the probability of the individual support values of A and B and not together. Lift explains the strength of a rule. **More the Lift more is the strength.** Let's say for A -> B, the lift value is 4. It means that if you buy A the chances of buying B is **4 times**. Let's get started with the Apriori Algorithm now and see how it works

Difference between Association and Recommendation

Association rules do not extract an individual's preference, rather find relationships between sets of elements of every distinct transaction. This is what makes them different than Collaborative filtering which is used in recommendation systems.
To understand it better take a look at below snapshot from amazon.com and you notice 2 headings "Frequently Bought Together" and the "Customers who bought this item also bought" on each product's info page.
"Frequently Bought Together" → Association
"Customers who bought this item also bought" → Recommendation

1 → Tea, Cake, Cold Drink
2 → Tea, Coffee, Cold Drink
3 → Eggs, Tea, Cold Drink
4 → Cake, Milk, Eggs
5 → Cake, Coffee, Cold Drink, Milk, Eggs

$$\text{Support} = \frac{\text{No. of transaction in which A appears}}{\text{Total no. of transactions}}$$

$$\text{Confidence } (A \rightarrow B) = \frac{\text{Support } (A \cup B)}{\text{Support } (A)}$$

Relative Support – The relative no. of transactions which contains an itemset relative to the total transactions.

Formula Relative support of tea = 3/5 =
Cake = 3/5
Cold Drink = 4/5
Milk = 2/5
Eggs = 3/5

Confidence – It is the probability that if a person buys an item A, then he will also buy an item B.

Confidence that if a person buy Tea, also buy cake : 1/3
= 0.2 = 20%

Why 1 because Tea & Cake occur together in 1 transaction
3 " there are 3 transactions in which tea is occurring.

Apriori Algorithm

Apriori algorithm assumes that any subset of a frequent itemset must be frequent. Its the algorithm behind Market Basket Analysis.

Apriori algorithm uses frequent itemsets to generate association rules. It is based on the concept that a subset of a frequent itemset must also be a frequent itemset. Frequent Itemset is an itemset whose support value is greater than a threshold value(support).

Let's say we have the following data of a store.

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

Iteration 1: Let's assume the support value is 2 and create the item sets of the size of 1 and calculate their support values.

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

C1

Itemset	Support
{1}	3
{2}	3
{3}	4
{4}	1
{5}	4

As you can see here, item 4 has a support value of 1 which is less than the min support value. So we are going to **discard {4}** in the upcoming iterations. We have the final Table F1.

C1

Itemset	Support
{1}	3
{2}	3
{3}	4
{4}	1
{5}	4

F1

Itemset	Support
{1}	3
{2}	3
{3}	4
{5}	4

Iteration 2: Next we will create itemsets of size 2 and calculate their support values. All the combinations of items set in F1 are used in this iteration.

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

Only Items present in F1

C2

Itemset	Support
{1,2}	1
{1,3}	3
{1,5}	2
{2,3}	2
{2,5}	3
{3,5}	3

F2

Itemset	Support
{1,3}	3
{1,5}	2
{2,3}	2
{2,5}	3
{3,5}	3

Itemsets having Support less than 2 are eliminated again. In this case **{1,2}.** Now, Let's understand what is pruning and how it makes Apriori one of the best algorithm for finding frequent itemsets.

Pruning: We are going to divide the itemsets in C3 into subsets and eliminate the subsets that are having a support value less than 2.

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

C3

Itemset	In F2?
{1,2,3}, {1,2}, {1,3}, {2,3}	NO
{1,2,5}, {1,2}, {1,5}, {2,5}	NO
{1,3,5},{1,5}, {1,3}, {3,5}	YES
{2,3,5}, {2,3}, {2,5}, {3,5}	YES

Iteration 3: We will discard **{1,2,3}** and **{1,2,5}** as they both contain **{1,2}.** This is the main highlight of the Apriori Algorithm.

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

F3

Itemset	Support
{1,3,5}	2
{2,3,5}	2

Iteration 4: Using sets of F3 we will create C4.

TID	Items
T1	1 3 4
T2	2 3 5
T3	1 2 3 5
T4	2 5
T5	1 3 5

F3

Itemset	Support
{1,3,5}	2
{2,3,5}	2

C3

Itemset	Support
{1,2,3,5}	1

Since the Support of this itemset is less than 2, we will stop here and the final itemset we will have is F3.
Note: Till now we haven't calculated the confidence values yet.

With F3 we get the following itemsets:

For I = {1,3,5}, subsets are {1,3}, {1,5}, {3,5}, {1}, {3}, {5}
For I = {2,3,5}, subsets are {2,3}, {2,5}, {3,5}, {2}, {3}, {5}

Applying Rules:

We will create rules and apply them on itemset F3. Now let's assume a minimum confidence value is **60%.**

For every subsets S of I, you output the rule

- S –> (I-S) (means S recommends I-S)
- if **support(I) / support(S) >= min_conf value**

<u>for set -{1,3,5} following are the rules generated:-</u>

Rule 1: {1,3} –> ({1,3,5} – {1,3}) means 1 & 3 –> 5

Confidence = support(1,3,5)/support(1,3) = 2/3 = **66.66% > 60%**

Hence Rule 1 is **Selected**

Rule 2: {1,5} –> ({1,3,5} – {1,5}) means 1 & 5 –> 3

Confidence = support(1,3,5)/support(1,5) = 2/2 = **100% > 60%**

Rule 2 is **Selected**

Rule 3: {3,5} –> ({1,3,5} – {3,5}) means 3 & 5 –> 1

Confidence = support(1,3,5)/support(3,5) = 2/3 = **66.66% > 60%**

Rule 3 is **Selected**

Rule 4: {1} –> ({1,3,5} – {1}) means 1 –> 3 & 5

Confidence = support(1,3,5)/support(1) = 2/3 = **66.66% > 60%**, hence Rule 4 is **Selected**

Rule 5: {3} –> ({1,3,5} – {3}) means 3 –> 1 & 5

Confidence = support(1,3,5)/support(3) = 2/4 = **50% <60%**

Rule 5 is **Rejected**

Rule 6: {5} –> ({1,3,5} – {5}) means 5 –> 1 & 3

Confidence = support(1,3,5)/support(5) = 2/4 = 50% < 60%

Rule 6 is **Rejected**

This is how you create rules in Apriori Algorithm and the same steps can be implemented for the itemset **{2,3,5}.**

Apriori Example 2:-

Here is a dataset consisting of six transactions. Each transaction is a combination of 0s and 1s, where 0 represents the absence of an item and 1 represents the presence of it.

Transaction ID	Grapes	Apple	Mango	Orange
1	1	1	1	1
2	1	0	1	1
3	0	0	1	1
4	0	1	0	0
5	1	1	1	1
6	1	1	0	1

Dataset

In order to find out interesting rules out of multiple possible rules from this small business scenario, we will be using the following matrices:

1. Support: Its the default popularity of an item. In mathematical terms, the support of item **A** is nothing but the ratio of transactions involving **A** to the total number of transactions.
Support(Grapes) = (Transactions involving Grapes)/(Total transaction)

Support(Grapes) = 0.666

2. Confidence: Likelihood that customer who bought both **A** and **B**. Its divides the number of transactions involving both **A** and **B** by the number of transactions involving **B**.
Confidence(A => B) = (Transactions involving both A and B)/(Transactions involving only A)

Confidence({Grapes, Apple} => {Mango}) = Support(Grapes, Apple, Mango)/ Support(Grapes, Apple)

= 2/6 / 3/6

= 0.667

3. Lift : Increase in the sale of **A** when you sell **B**.
Lift(A => B) = Confidence(A, B) / Support(B)

Lift ({Grapes, Apple} => {Mango}) = 1

So, likelihood of a customer buying both **A** and **B** together is ‘lift-value’ times more than the chance if purchasing alone.

- **Lift (A => B)** = 1 means that there is no correlation within the itemset.

- **Lift (A => B)** > 1 means that there is a positive correlation within the itemset, i.e., products in the itemset, **A**, and **B**, are more likely to be bought together.
- **Lift (A => B)** < 1 means that there is a negative correlation within the itemset, i.e., products in itemset, **A**, and **B**, are unlikely to be bought together.

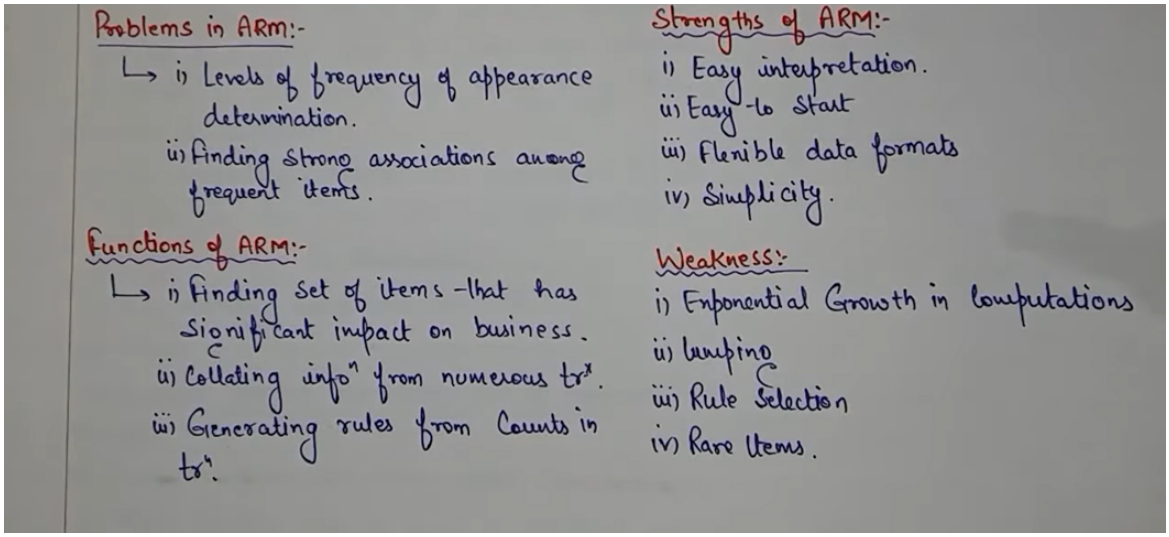
Association Rule-based algorithms are viewed as a two-step approach:

1. **Frequent Itemset Generation:** Find all frequent item-sets with support >= pre-determined min_support count
2. **Rule Generation:** List all Association Rules from frequent item-sets. Calculate Support and Confidence for all rules. Prune rules that fail min_support and min_confidence thresholds.

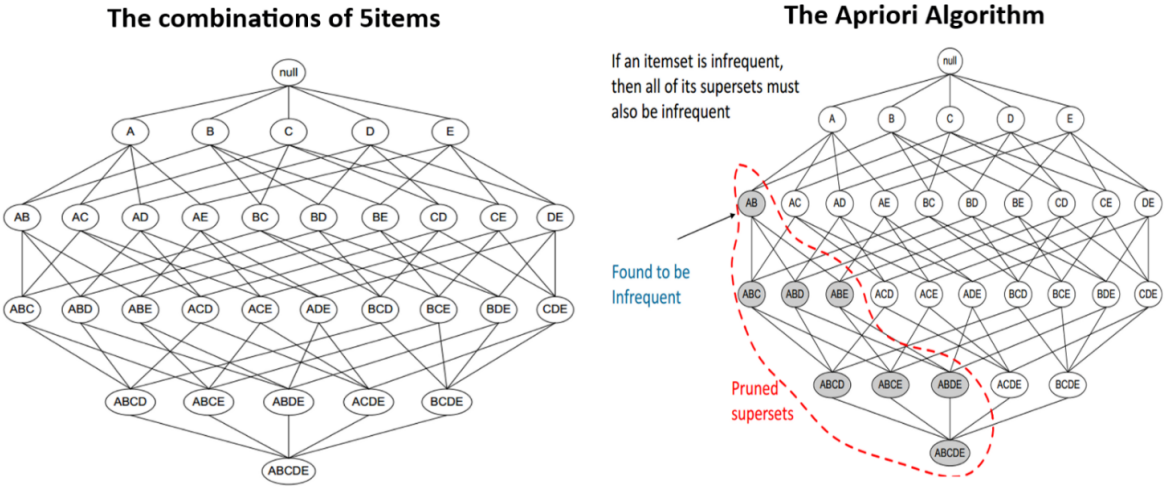
Limitation of Apriori algorithm

Frequent Itemset Generation is the most computationally expensive step because the algorithm scans the database too many times, which reduces the overall performance. Due to this, the algorithm assumes that the database is Permanent in the memory.

Also, both the time and space complexity of this algorithm are very high: **O(2^{|D|}),** thus exponential, where |D| is the horizontal width (the total number of items) present in the database.



Optimizing Apriori algorithm



Transaction reduction

We can optimize the existing apriori algorithm by which it will take less time and also works with less memory using these methods:

- **Hash-based itemset counting:** A k-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent.
- **Transaction reduction:** A transaction that does not contain any frequent k-itemsets useless in subsequent scans.
- **Partitioning:** An itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB.
- **Sampling:** Mining on a subset of given data, lower support threshold + a method to determine the completeness.
- **Dynamic itemset counting:** add new candidate itemsets only when all of their subsets are estimated to be frequent.

a) Hash Based Techniques

- Hash table is used as data structure.
- First iteration is required to count support of each itemset.
- From second iteration, efforts are made to enhance execution of Apriori by utilizing hash table concept.
- Hash table minimizes the number of itemset generated in second iteration.
- In second iteration i.e. 2-itemset generation, for every combination of two item, we map them into the diverse bucket of hash table structure and increment the bucket count.
- If count of bucket is less than min. sup. count, we remove them from candidate sets.

TID	List of Items
T1	I1, I2, I5
T2	I2, I4
T3	I2, I3
T4	I1, I2, I4
T5	I1, I3
T6	I2, I3
T7	I1, I3
T8	I1, I2, I3, I5
T9	I1, I2, I3

Min. Support Count=3

C1

Itemset	Support Count
I1	6
I2	7
I3	6
I4	2
I5	2

Hash Function

Itemset	Count	Hash Function
I1, I2	4	[1*10+2] mod 7=**5**
I1, I3	4	[1*10+3] mod 7=**6**
I1, I4	1	[1*10+4] mod 7=**0**
I1, I5	2	[1*10+5] mod 7=**1**
I2, I3	4	[2*10+3] mod 7=**2**
I2, I4	2	[2*10+4] mod 7=**3**
I2, I5	2	[2*10+5] mod 7=**4**
I3, I4	0	--
I3, I5	1	[3*10+5] mod 7=**0**
I4, I5	0	--

Order of Items I1=1, I2=2, I3=3, I4=4, I5=5

H(x, y)=((Order of First)* 10+(Order of Second))mod 7

Hash Table Structure to generate L2

Bucket address	0	1	2	3	4	5	6
Bucket Count	**2**	**2**	4	**2**	**2**	4	4
Bucket Contents	{I1-I4}-1 {I3-I5}-1	{I1-I5}-2	{I2-I3}-4	{I2-I4}-2	{I2-I5}-2	{I1-I2}-4	{I1-I3}-4
L2	No	No	Yes	No	No	Yes	Yes

Advantages:

- Reduce the number of scans
- Remove the large candidates that cause high Input/output cost

b) Transaction Reduction

Transaction that does not contain any frequent k-itemsets cannot contain any frequent (k+1)-itemsets. Therefore, such a transaction can be marked or removed from further consideration

Trans.	Items
T1	I1, I2, I5
T2	I2, I3, I4
T3	I3, I4
T4	I1, I2, I3, I4

	I1	I2	I3	I4	I5
T1	1	1	0	0	1
T2	0	1	1	1	0
T3	0	0	1	1	0
T4	1	1	1	1	0

	I1	I2	I2	I4	I5
T1	1	1	0	0	1
T2	0	1	1	1	0
T3	0	0	1	1	0
T4	1	1	1	1	0

Minimum Support Count=2

	I1	I2	I2	I4
T1	1	1	0	0
T2	0	1	1	1
T3	0	0	1	1
T4	1	1	1	1

Trans.	Items
T1	I1, I2, I5
T2	I2, I3, I4
T3	I3, I4
T4	I1, I2, I3, I4

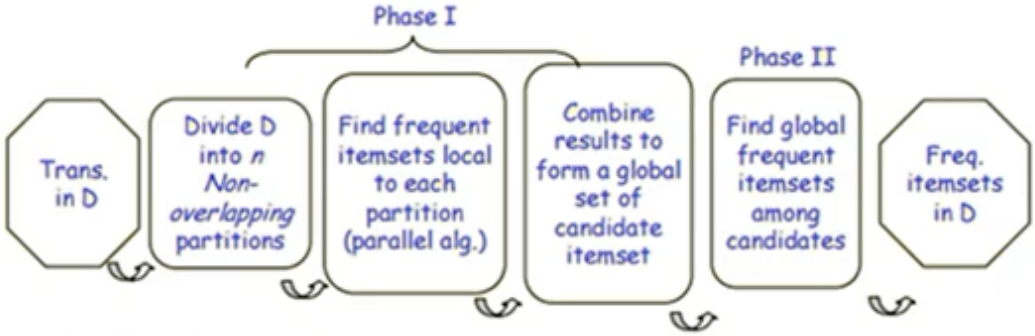
	I1,I2	I1,I3	I1,I4	I2,I3	I2,I4	I3,I4
T1	1	0	0	0	0	0
T2	0	0	0	1	1	1
T3	0	0	0	0	0	1
T4	1	1	1	1	1	1

	I1,I2	I2,I3	I2,I4	I3,I4
T2	0	1	1	1
T4	1	1	1	1

	I2,I3, I4
T2	1
T4	1

c) Partitioning

Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB (2 DB Scan)



	I1	I2	I3	I4	I5
T1	1	0	0	0	1
T2	0	1	0	1	0
T3	0	0	0	1	1
T4	0	1	1	0	0
T5	0	0	0	0	1
T6	0	1	1	1	0

Database is divided into three partitions.

Each having two transactions with support of 20%

Trans.	Itemset	First Scan	Second Scan	Shortlisted
		Support =20% **Min. sup=1**	**Support =20%** **Min. sup=2**	
T1	I1, I5	I1-1, I2-1, I4-1, I5-1	I1-1, I2-3	I2-3, I3-2
T2	I2, I4	{I1,I5}-1 {I2, I4}-1	I3-2. I4-3	I4-3 ,I5-3
T3	I4, I5	I2-1, I3-1, I4-1, I5-1	I5-3, {I1, I5}-1	{I2, I4}-2
T4	I2, I3	{I4,I5}-1,{I2, I3}-1	{I2, I4}-2, {I4, I5}-1	{I2, I3}-2
T5	I5	I2-1, I3-1,I4-1, I5-1	{I2, I3}-2, {I3,I4}-1	
T6	I2, I3, I4	{I2, I3}-1, {I2,I4}-1 {I3, I4}-1 {I2, I3, I4}-1	{I2, I3, I4}-1	

d) Dynamic Itemset Counting

It is an algorithm which reduces the number of passes made over the data while keeping the number of itemsets which are counted in any pass relatively low.

This technique can add new candidate itemsets at any marked start point of the database during the scanning of the database.

e) Sampling

The basic idea is to pick up a random sample S of the given data D, and then search for frequent itemsets in S instead of D.

It may be possible to lose a global frequent itemset. This can be reduced by lowering the min_sup.

There is trade off some degree of accuracy against efficiency.

Shortcomings Of Apriori Algorithm

1. Using Apriori needs a generation of candidate itemsets. These itemsets may be large in number if the itemset in the database is huge.
2. Apriori needs multiple scans of the database to check the support of each itemset generated and this leads to high costs.

These shortcomings can be overcome using the FP growth algorithm.

Frequent Pattern Growth Algorithm

This algorithm is an improvement to the Apriori method. A frequent pattern is generated without the need for candidate generation. FP growth algorithm represents the database in the form of a tree called a frequent pattern tree or FP tree.

This tree structure will maintain the association between the itemsets. The database is fragmented using one frequent item. This fragmented part is called “pattern fragment”. The itemsets of these fragmented patterns are analyzed. Thus with this method, the search for frequent itemsets is reduced comparatively.

FP Tree

Frequent Pattern Tree is a tree-like structure that is made with the initial itemsets of the database. The purpose of the FP tree is to mine the most frequent pattern. Each node of the FP tree represents an item of the itemset.

The root node represents null while the lower nodes represent the itemsets. The association of the nodes with the lower nodes that is the itemsets with the other itemsets are maintained while forming the tree.

Frequent Pattern Algorithm Steps

The frequent pattern growth method lets us find the frequent pattern without candidate generation.

Let us see the steps followed to mine the frequent pattern using frequent pattern growth algorithm:

#1) The first step is to scan the database to find the occurrences of the itemsets in the database. This step is the same as the first step of Apriori. The count of 1-itemsets in the database is called support count or frequency of 1-itemset.

#2) The second step is to construct the FP tree. For this, create the root of the tree. The root is represented by null.

#3) The next step is to scan the database again and examine the transactions. Examine the first transaction and find out the itemset in it. The itemset with the max count is taken at the top, the next itemset with lower count and so on. It means that the branch of the tree is constructed with transaction itemsets in descending order of count.

#4) The next transaction in the database is examined. The itemsets are ordered in descending order of count. If any itemset of this transaction is already present in another branch (for example in the 1st transaction), then this transaction branch would share a common prefix to the root.

This means that the common itemset is linked to the new node of another itemset in this transaction.

#5) Also, the count of the itemset is incremented as it occurs in the transactions. Both the common node and new node count is increased by 1 as they are created and linked according to transactions.

#6) The next step is to mine the created FP Tree. For this, the lowest node is examined first along with the links of the lowest nodes. The lowest node represents the frequency pattern length 1. From this, traverse the path in the FP Tree. This path or paths are called a conditional pattern base.

Conditional pattern base is a sub-database consisting of prefix paths in the FP tree occurring with the lowest node (suffix).

#7) Construct a Conditional FP Tree, which is formed by a count of itemsets in the path. The itemsets meeting the threshold support are considered in the Conditional FP Tree.

#8) Frequent Patterns are generated from the Conditional FP Tree.

Example Of FP-Growth Algorithm

Support threshold=50%, Confidence= 60%

Table 1

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4
T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
T6	I1,I2,I3,I4

Solution:

Support threshold=50% => 0.5*6= 3 => min_sup=3

1. Count of each item

Table 2

Item	Count
I1	4
I2	5
I3	4
I4	4
I5	2

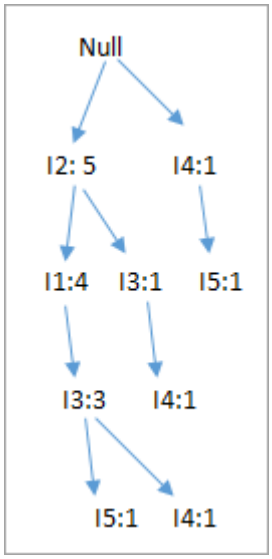
2. Sort the itemset in descending order.

Table 3

Item	Count
I2	5
I1	4
I3	4
I4	4

3. Build FP Tree

1. Considering the root node null.
2. The first scan of Transaction T1: I1, I2, I3 contains three items {I1:1}, {I2:1}, {I3:1}, where I2 is linked as a child to root, I1 is linked to I2 and I3 is linked to I1.
3. T2: I2, I3, I4 contains I2, I3, and I4, where I2 is linked to root, I3 is linked to I2 and I4 is linked to I3. But this branch would share I2 node as common as it is already used in T1.
4. Increment the count of I2 by 1 and I3 is linked as a child to I2, I4 is linked as a child to I3. The count is {I2:2}, {I3:1}, {I4:1}.
5. T3: I4, I5. Similarly, a new branch with I5 is linked to I4 as a child is created.
6. T4: I1, I2, I4. The sequence will be I2, I1, and I4. I2 is already linked to the root node, hence it will be incremented by 1. Similarly I1 will be incremented by 1 as it is already linked with I2 in T1, thus {I2:3}, {I1:2}, {I4:1}.
7. T5:I1, I2, I3, I5. The sequence will be I2, I1, I3, and I5. Thus {I2:4}, {I1:3}, {I3:2}, {I5:1}.
8. T6: I1, I2, I3, I4. The sequence will be I2, I1, I3, and I4. Thus {I2:5}, {I1:4}, {I3:3}, {I4 1}.

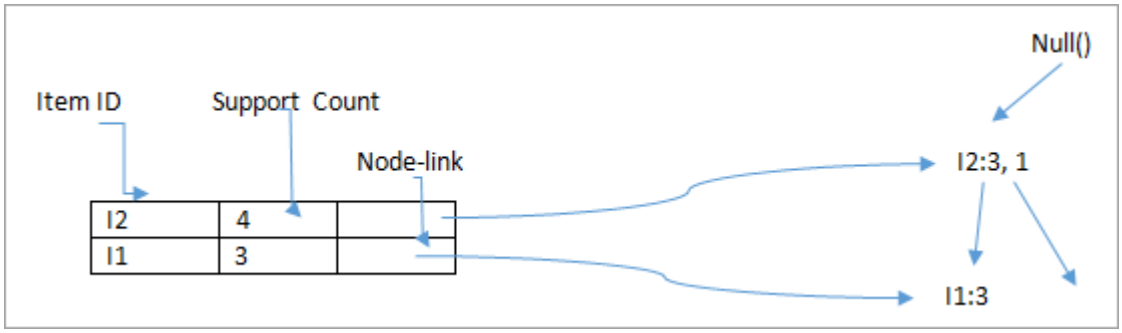


4. Mining of FP-tree is summarized below:

1. The lowest node item I5 is not considered as it does not have a min support count, hence it is deleted.
2. The next lower node is I4. I4 occurs in 2 branches , {I2,I1,I3:,I41},{I2,I3,I4:1}. Therefore considering I4 as suffix the prefix paths will be {I2, I1, I3:1}, {I2, I3: 1}. This forms the conditional pattern base.
3. The conditional pattern base is considered a transaction database, an FP-tree is constructed. This will contain {I2:2, I3:2}, I1 is not considered as it does not meet the min support count.
4. This path will generate all combinations of frequent patterns : {I2,I4:2},{I3,I4:2}, {I2,I3,I4:2}
5. For I3, the prefix path would be: {I2,I1:3},{I2:1}, this will generate a 2 node FP-tree : {I2:4, I1:3} and frequent patterns are generated: {I2,I3:4}, {I1:I3:3}, {I2,I1,I3:3}.
6. For I1, the prefix path would be: {I2:4} this will generate a single node FP-tree: {I2:4} and frequent patterns are generated: {I2, I1:4}.

Item	Conditional Pattern Base	Conditional FP-tree	Frequent Patterns Generated
I4	{I2,I1,I3:1},{I2,I3:1}	{I2:2, I3:2}	{I2,I4:2},{I3,I4:2},{I2,I3,I4:2}
I3	{I2,I1:3},{I2:1}	{I2:4, I1:3}	{I2,I3:4}, {I1:I3:3}, {I2,I1,I3:3}
I1	{I2:4}	{I2:4}	{I2,I1:4}

The diagram given below depicts the conditional FP tree associated with the conditional node I3.



Advantages Of FP Growth Algorithm

1. This algorithm needs to scan the database only twice when compared to Apriori which scans the transactions for each iteration.
2. The pairing of items is not done in this algorithm and this makes it faster.
3. The database is stored in a compact version in memory.
4. It is efficient and scalable for mining both long and short frequent patterns.

Disadvantages Of FP-Growth Algorithm

1. FP Tree is more cumbersome and difficult to build than Apriori.
2. It may be expensive.
3. When the database is large, the algorithm may not fit in the shared memory.

FP Growth vs Apriori

FP Growth	Apriori
Pattern Generation	
FP growth generates pattern by constructing a FP tree	Apriori generates pattern by pairing the items into singletons, pairs and triplets.
Candidate Generation	
There is no candidate generation	Apriori uses candidate generation
Process	
The process is faster as compared to Apriori. The runtime of process increases linearly with increase in number of itemsets.	The process is comparatively slower than FP Growth, the runtime increases exponentially with increase in number of itemsets
Memory Usage	
A compact version of database is saved	The candidates combinations are saved in memory

ECLAT Algo

TID	Items
T1	I1, I3, I4
T2	I2, I3, I5, I6
T3	I1, I2, I3, I5
T4	I2, I5
T5	I1, I3, I5

↓ In Vertical Format

Items	Transaction IDs
I1	T1, T3, T5
I2	T2, T3, T4
I3	T1, T2, T3, T5
I4	T1
I5	T2, T3, T4, T5
I6	T2

	T1	T2	T3	T4	T5
I1	1	0	1	0	1
I2	0	1	1	1	0
I3	1	1	1	0	1
I4	1	0	0	0	0
I5	0	1	1	1	1
I6	0	1	0	0	0

Sup count

I1 → 3
I2 → 3
I3 → 4
I4 → 1
I5 → 4
I6 → 1

I1 – 5
I2 – 3
I3 – 4
I4 – 4

Create Frequent Itemsets with 2 items

{I1, I2}	T3	1	✗
I1, I3	T1, T3, T5	3	
I1, I5	T3, T5	2	
I2, I3	T2, T3	2	
I2, I5	T2, T3, T4	3	
I3, I5	T2, T3, T5	3	

Create Frequent Itemsets with 3 items

(I1, I3, I5) (I1, I2, I3) (I1, I2, I5) (I2, I3, I5)

On above itemsets we will perform pruning to remove any itemset that has a subset that is not frequent itemset.

(I1, I3, I5)	(I1, I3) (I1, I5) (I3, I5)	Yes
(I1, I2, I3)	(I1, I2) (I1, I3) (I2, I3)	No
(I1, I2, I5)	(I1, I2) (I1, I5) (I2, I5)	No
(I2, I3, I5)	(I2, I3) (I2, I5) (I3, I5)	Yes

Hence

		SC
(I1, I3, I5)	T2, T3	2
(I1, I3, I5)	T3, T5	2

(a) Frequent Itemsets with 4 items

(I1, I2, I3, I5)

↓

Pruning — (I1, I2, I5) & (I1, I2, I3) are not frequent itemsets. Hence the process stops here.

Finally

I1	3
I2	3
I3	4
I5	4
(I1, I3)	3
(I1, I5)	2
(I2, I3)	2
(I2, I5)	2
(I3, I5)	3
(I2, I3, I5)	2
(I1, I3, I5)	2

I1 → I3 , I3 → I1

I2 → I3 ^ I5 , I3 → I2 ^ I5, I5 → I2 ^ I3

ECLAT(mining frequent itemsets using the vertical data format)

The above method, Apriori and FP growth, mine frequent itemsets using horizontal data format. ECLAT is a method of mining frequent itemsets using the vertical data format. It will transform the data in the horizontal data format into the vertical format.

For Example, Apriori and FP growth use:

Transaction	List of items
T1	I1,I2,I3
T2	I2,I3,I4
T3	I4,I5
T4	I1,I2,I4
T5	I1,I2,I3,I5
T6	I1,I2,I3,I4

The ECLAT will have the format of the table as:

Item	Transaction Set
I1	{T1,T4,T5,T6}
I2	{T1,T2,T4,T5,T6}
I3	{T1,T2,T5,T6}
I4	{T2,T3,T4,T5}
I5	{T3,T5}

This method will form 2-itemsets, 3 itemsets, k itemsets in the vertical data format. This process with k is increased by 1 until no candidate itemsets are found. Some optimizations techniques such as diffset are used along with Apriori.

This method has an advantage over Apriori as it does not require scanning the database to find the support of k+1 itemsets. This is because the Transaction set will carry the count of occurrence of each item in the transaction (support). The bottleneck comes when there are many transactions taking huge memory and computational time for intersecting the sets.

Conclusion

The Apriori algorithm is used for mining association rules. It works on the principle, "the non-empty subsets of frequent itemsets must also be frequent". It forms k-itemset candidates from (k-1) itemsets and scans the database to find the frequent itemsets.

Frequent Pattern Growth Algorithm is the method of finding frequent patterns without candidate generation. It constructs an FP Tree rather than using the generate and test strategy of Apriori. The focus of the FP Growth algorithm is on fragmenting the paths of the items and mining frequent patterns.

From Association Mining to Correlation Analysis

Most association rule mining algorithms employ a support-confidence framework. Often, many interesting rules can be found using low support thresholds. Although minimum support and confidence thresholds help weed out or exclude the exploration of a good number of uninteresting rules, many rules so generated are still not interesting to the users. Unfortunately, this is especially true when mining at low support thresholds or mining for long patterns. This has been one of the major bottlenecks for successful application of association rulemining

1)Strong Rules Are Not Necessarily Interesting:

An Example Whether or not a rule is interesting can be assessed either subjectively or objectively. Ultimately, only the user can judge if a given rule is interesting, and this judgment, being subjective, may differ from one user to another. However, objective interestingness measures, based on the statistics —behind‖ the data, can be used as one step toward the goal of weeding out uninteresting rules from presentation to the user.

The support and confidence measures are insufficient at filtering out uninteresting association rules. To tackle this weakness, a correlation measure can be used to augment the support-confidence framework for association rules. This leads to correlation rules of the form.

That is, a correlation rule is measured not only by its support and confidence but also by the correlation between itemsets A and B. There are many different correlation measures from which to choose. In this section, we study various correlation measures to determine which would be good for mining large data sets

As we can see the support-confidence framework can be misleading;

• it can identify a rule A=>B as interesting (strong) when, in fact the occurrence of A might not imply the occurrence of B

• Correlation Analysis provides an alternative framework for finding interesting relationships, or to improve understanding of meaning of some association rules (a lift of an association rule)

Definition: Two item sets A and B are independent (the occurrence of A is independent of the occurrence of item set B) iff probability P fulfills the condition:-

$$P(A \cup B) = P(A) \cdot P(B)$$

Otherwise A and B are dependent or correlated

- The measure of correlation, or correlation between A and B is given by the formula:

$$Corr(A,B) = P(A \cup B) / P(A)P(B)$$

- **corr(A,B) >1** means that A and B are positively correlated i.e. the occurrence of one implies the occurrence of the other (can be called as strongly correlated)
- **corr(A,B) < 1** means that the occurrence of A is negatively correlated with B
- or discourages the occurrence of B
- **corr(A,B) =1** means that A and B are independent

The correlation formula can be re-written as :- Corr(A,B) = Supp(A=>B)= P(AUB)

Conf(A=>B)= P(B|A), i.e. Conf(A=>B)= corr(A,B) P(B), So correlation, support and confidence are all different, but the correlation provides an extra information about the association rule (A=>B).

We say that the correlation corr(A,B) provides the LIFT of the association rule (A=>B), i.e. A is said to increase or to LIFT the likelihood of B by the factor of the value returned by the formula for corr(A,B)